

CS3012E Artificial Intelligence - Course Project Report

Imbalanced Text Classification Study

Team Details

- Section: CS02
 - Team Members: 1. Malepati Venkata Gowrish - B230673CS
2. Renish Dsouza - B231198CS
3. Rudrakshala Gowrinath - B230530CS
4. Akkala Pavan Reddy - B230152CS
5. Putta Shyam Sekhar - B230509CS
 - Submission Date: 6 April 2026
-

1. Introduction

The project is there to do binary text classification on an imbalanced dataset. The goal is to compare model behaviour in two setups:

- Without imbalance handling
- With imbalance handling

Three model families were implemented as required in the problem statement:

- Logistic Regression
- Random Forest
- Neural Network (fixed 64-32-16 hidden architecture)

Evaluation is done using class-aware metrics such as precision, recall, F1, ROC-AUC, and PR-AUC.

2. Dataset Description

- Dataset: Disaster Tweets
- Source: <https://www.kaggle.com/datasets/vstepanenko/disaster-tweets/data>
- Target:
 - Class 0: Non-disaster tweet
 - Class 1: Disaster tweet
- Feature:
 - tweet text (vectorized using TF-IDF)

Observed Imbalance (Test Split)

From saved notebook outputs, the test split has 2274 samples:

Class	Meaning	Support in Test Set
0	Non-disaster	1878
1	Disaster	396

This confirms minority representation for disaster tweets and motivates imbalance handling.

3. Preprocessing

Preprocessing and feature engineering used in the provided notebooks/scripts:

1. Row filtering: selected `text` and `target`, removed missing values where applicable.
2. Text cleaning (logistic regression pipeline):
 - lowercase conversion,
 - URL removal,
 - removal of non-alphabetic characters.
3. Feature extraction:
 - TF-IDF vectorization (`max_features` up to 5000 in some experiments),
 - unigram/bigram settings for logistic regression experiment.

The resulting vectors are used as inputs for all three model families.

4. Methodology

4.1 Logistic Regression

- Vectorized input: TF-IDF
- Tuned components:
 - regularization type (`l1`, `l2`)
 - regularization strength (`C`)
 - solver (`liblinear`)
 - decision threshold (0.3, 0.4, 0.5, 0.6)
- Additional imbalance-aware variants:
 - class weighting (`class_weight='balanced'`)
 - SMOTE on training vectors

4.2 Random Forest

- Vectorized input: TF-IDF
- Tuned components included:
 - number of trees (`n_estimators`)
 - maximum depth (`max_depth`)
 - minimum samples split (`min_samples_split`)
 - minimum samples leaf (`min_samples_leaf`)
- Imbalance-aware variants used:
 - SMOTE
 - class weighting

4.3 Neural Network (Fixed Architecture)

Architecture (kept fixed as required):

- Input layer
- Dense(64)
- Dense(32)

- Dense(16)
- Sigmoid output

Tuned components across experiments:

- activation (`tanh` and `relu` variants)
- optimizer (`adam`)
- epochs (14 for without imbalance handling, 30 for with imbalance handling)
- dropout (0.3 for with imbalance handling)
- L2 regularization (0.01 for with imbalance handling)
- learning-rate scheduling (used in with imbalance handling)
- early stopping (used in with imbalance handling)
- decision threshold (used in both with imbalance handling and without imbalance handling)

Imbalance handling used in Part B:

- SMOTE on training split
- class weighting during training

5. Experimental Setup

- Language: Python
- Main files used:
 - `ai-sem-6-course-project.ipynb`
 - `WITHOUT_IMBALANCE_HANDLING_AI_PROJECT (1).ipynb`
 - `WITH_IMBALANCE_HANDLING_AI_PROJECT.ipynb`
 - `lr.py`
- Data split:
 - train/test split (`20% of data`)
 - validation split (`25% of train data from train/test split`)

Note: the saved notebook outputs use slightly different preprocessing/splitting choices across experiments. The metrics below are therefore reported exactly as saved in each notebook, without forcing them into a single synthetic split.

Part A - Without Imbalance Handling

Baseline models trained directly on original training distribution.

Part B - With Imbalance Handling

Techniques applied:

1. Class weighting
2. SMOTE

6. Results

6.1 Logistic Regression (from `ai-sem-6-course-project.ipynb`)

Variant	Class 0 Precision	Class 0 Recall	Class 0 F1	Class 1 Precision	Class 1 Recall	Class 1 F1	Accuracy
Base Model	0.86	0.99	0.92	0.84	0.30	0.45	0.8597
Tuned Model	0.90	0.96	0.93	0.76	0.54	0.63	0.8817
Class Weight Model	0.94	0.88	0.91	0.59	0.74	0.65	0.8549
SMOTE Model	0.93	0.87	0.90	0.57	0.73	0.64	0.8478

Notes:

- Threshold sweeps were evaluated at 0.3, 0.4, 0.5, and 0.6.
- The best disaster-class recall among the logistic variants came from class weighting.

6.2 Random Forest (from [ai-sem-6-course-project.ipynb](#))

Variant	Class 0 Precision	Class 0 Recall	Class 0 F1	Class 1 Precision	Class 1 Recall	Class 1 F1	Accuracy
Baseline RF	0.90	0.98	0.94	0.81	0.48	0.60	0.89
RF + SMOTE	0.91	0.96	0.93	0.75	0.55	0.64	0.89
RF + Class Weight	0.89	0.98	0.93	0.82	0.43	0.56	0.88

Observation: SMOTE improved RF ROC-AUC and macro F1 compared with the baseline.

6.3 Neural Network (from dedicated notebooks)

Variant	Class 0 Precision	Class 0 Recall	Class 0 F1	Class 1 Precision	Class 1 Recall	Class 1 F1	Accuracy
Without imbalance handling	0.92	0.95	0.93	0.71	0.61	0.65	0.89
With imbalance handling	0.92	0.95	0.94	0.74	0.62	0.67	0.90

6.4 Cross-Model Comparison

Metric	Logistic Regression Best	Random Forest Best	Neural Network Best
Accuracy	0.8817	0.89	0.90
Class 1 Precision	0.76	0.82	0.74
Class 1 Recall	0.74	0.55	0.62
Class 1 F1	0.65	0.64	0.67
Macro F1	0.78	0.79	0.81
Weighted F1	0.87	0.88	0.89
ROC-AUC	0.8985	0.7568	0.8968
PR-AUC	N/A	N/A	0.7497

Key takeaways:

- Logistic Regression achieved the best ROC-AUC among the models saved in the notebooks.
- Neural Network with imbalance handling achieved the best overall balance of macro F1, weighted F1, and PR-AUC.

- Random Forest benefited from SMOTE, but remained weaker on ROC-AUC than the other two model families.
-

7. Analysis and Discussion

1. Accuracy alone is insufficient for imbalanced data.
 - High overall accuracy can coexist with weak minority-class recall.
 2. Logistic Regression with class weighting improved disaster recall to 0.74, which is the strongest minority-class recall among the logistic variants.
 3. Neural-network imbalance handling improved the balanced metrics end to end.
 - Disaster-class precision improved from 0.71 to 0.74.
 - Disaster-class recall improved from 0.61 to 0.62.
 - Macro F1 and PR-AUC both increased.
 1. For Random Forest, SMOTE performed better than class weighting in this setup.
 - ROC-AUC increased from 0.7269 to 0.7568.
 - Macro F1 also improved from 0.77 to 0.79.
 1. PR-AUC is the most useful metric among the ones explicitly measured for the neural network because it reflects minority-class retrieval under imbalance.
-

8. Conclusion

The experiments confirm that imbalance handling is necessary for this task. Across models, imbalance-aware strategies improved class-sensitive metrics, especially for the disaster class.

Final takeaways:

- Best overall consistency was observed with imbalance-aware neural-network training (SMOTE + class weighting), which improved accuracy, macro F1, ROC-AUC, and PR-AUC compared with Part A.
- Random Forest benefited from SMOTE more than from class weighting in this setup.
- Logistic Regression remained competitive and interpretable, but required threshold and regularization tuning for better minority performance.

Limitations and Future Work

- Standardize train/test protocol across all notebooks for perfectly matched comparisons.
 - Add cross-validation and confidence intervals.
 - Evaluate transformer-based encoders (for example, BERT) for richer semantic representation.
 - Add probability calibration and cost-sensitive threshold optimization for deployment.
-

Appendix: Reproducibility Artifacts

- Notebook: `ai-sem-6-course-project.ipynb`
- Notebook: `WITHOUT_IMBALANCE_HANDLING_AI_PROJECT (1).ipynb`
- Notebook: `WITH_IMBALANCE_HANDLING_AI_PROJECT.ipynb`
- Script: `lr.py`
- Requirements: `requirements.txt`